

target AVR microcontroller.

Currently, all the ATmega AVR microcontroller devices, except ATmega640/1280/1281/2560/2561, have Flash size of up to 128 kB. Writing (burning the code into) the Flash of ATmega640/1280/1281/2560/2561 requires extended addresses. Also, as these devices have a higher pin count and come in TQFP package, they will not fit into the programmer board described here. This AVR programmer works fine with ATmega8515 and ATmega16/32.

If the user is programming using IAR Embedded Workbench, the steps for generating a raw binary file (*.bin) file are:

Goto Project→Options→Linker→Extra Output→Tick 'Generate Extra Output File'→Tick 'Override Default'→Select 'Output Format' as 'Raw-Binary' (refer Fig. 7).

A VB.NET program writes the raw binary file starting from address '0000' of the flash memory. So when writing a code which does not start from '0000' address, the user needs to take care by editing the linker (*.xcl) file before compiling the AVR

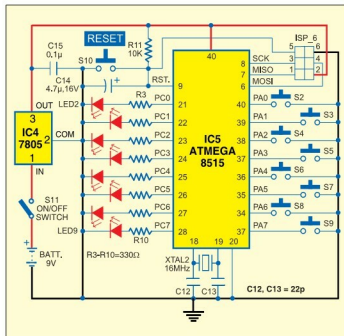


Fig. 8: LED switching circuit

source code.

A sample code for LED switching (led_sw.c) is given at the end of this article to test the working of the AVR programmer board. This code has to be burnt into the target device. Eight LEDs are connected to Port C

of the microcontroller (IC5) with their controlling switches connected to Port A. Initially, when IC5 is programmed successfully, all the LEDs glow. Now if you press any of the switches, the corresponding LED should go off. Fig. 8 shows the circuit for LED switching.

E F Y note. The source codes

AVR910.C, LED_SW.C and ADU_ISP are available on www.efymag.com website.

Arun Dayal Udai is an assistant professor in Department of Mechanical Engineering at Birla Institute of Technology, Mesra, and Brij Mohan is a project leader at Mphasis Ltd, Bengaluru

LED_SW.C

```

/* led_sw.c */

#include <iostream.h>
#include <intrinsics.h>

#define LED0 PORTC0 //Connected to PORTC
#define LED1 PORTC1 //Connected to PORTC
#define LED2 PORTC2 //Connected to PORTC
#define LED3 PORTC3 //Connected to PORTC
#define LED4 PORTC4 //Connected to PORTC
#define LED5 PORTC5 //Connected to PORTC
#define LED6 PORTC6 //Connected to PORTC
#define LED7 PORTC7 //Connected to PORTC

#define SW0 PINA0
#define SW1 PINA1 //All Switches
#define SW2 PINA2
#define SW3 PINA3
#define SW4 PINA4
#define SW5 PINA5
#define SW6 PINA6
#define SW7 PINA7

#define SETBIT(ADDRESS, BIT) (ADDRESS
|= (1 << BIT))
#define CLEARBIT(ADDRESS, BIT) (ADDRESS
&= ~(1 << BIT))

#define CHECKBIT(ADDRESS, BIT) ((AD-
DRESS & (1 << BIT)))

_C_task void main(void)
{
    //Set all LEDs pins as Output
    DDRC = 0xFF;
    //Enable pull-ups on all
    switches
    PORTA = 0xFF;

    while(1)
    {
        if (CHECKBIT(PINA, SW0))
        //Press to Low for Switches
        CLEARBIT(PORTC, LED0);
        else
        SETBIT(PORTC, LED0);

        if (CHECKBIT(PINA, SW1))
        CLEARBIT(PORTC, LED1);
        else
        SETBIT(PORTC, LED1);

        if (CHECKBIT(PINA, SW2))
        CLEARBIT(PORTC, LED2);
        else
        SETBIT(PORTC, LED2);

        if (CHECKBIT(PINA, SW3))
        CLEARBIT(PORTC, LED3);
        else
        SETBIT(PORTC, LED3);

        if (CHECKBIT(PINA, SW4))
        CLEARBIT(PORTC, LED4);
        else
        SETBIT(PORTC, LED4);

        if (CHECKBIT(PINA, SW5))
        CLEARBIT(PORTC, LED5);
        else
        SETBIT(PORTC, LED5);

        if (CHECKBIT(PINA, SW6))
        CLEARBIT(PORTC, LED6);
        else
        SETBIT(PORTC, LED6);

        if (CHECKBIT(PINA, SW7))
        CLEARBIT(PORTC, LED7);
        else
        SETBIT(PORTC, LED7);
    }
}

```